GROWTH HACKER

Growth Hacker Agent

The Complete Plain-English Guide

What it is, how it was made, how to set it up and run it, everything it can do, and every tool it connects to.

Iksula Agentic Org · iKshana assembly line
Plain-English guide · no technical background needed
Version 2026-06-23

Contents

1 What it is, and where it fits in the bigger picture

The one-line version

The **Growth Hacker** is the agent (a piece of software that follows written instructions to do a job) that **takes Iksula's already-approved content and actually gets it out into the world** — posting to LinkedIn, X (formerly Twitter), YouTube, the blog, and Telegram. It then watches who reacts (likes, comments, replies, clicks, form fills), runs the small tests the plan asked for, and **hands the names of interested people over to the next agent (Lead Gen)**.

Think of a newsroom. Reporters write the stories and editors approve them. The Growth Hacker is the **distribution desk** — it decides when each story goes out, formats it for each outlet, sends it, and then reads the mailbag to see which stories landed. It does **not** write the stories and it does **not** change what they say. As the skill file puts it: *"Execute the plan — never re-plan."*

Who it is for

- **Iksula's operators** (the people who run the day-to-day, like Vishal) who need posts published on schedule, native to each platform, without re-arguing the message.
- **The business owner / founder (DJ)** who wants Iksula's marketing to run like an assembly line, where every step has one clear owner and nothing dangerous can happen by accident.
- It is **not** a tool the public touches. It works behind the scenes, between the people who make content and the people who chase sales.

What it does — and just as importantly, what it does NOT do

What it **does** (its job, in the skill's words, is the "distribution executor and growth operator" on the "organic/online, 1-to-many" channel — meaning broadcasting to many people at once, not one-on-one):

- **Publishes** each approved asset to its channel, on schedule, in that channel's native shape (a long article becomes a thread or carousel where that fits — *reshaped, never rewritten*).
- **Runs the planned experiments** — small A/B tests (showing two versions and seeing which wins) on things like the hook, the post time, the audience slice, or the call-to-action. Never on the core message.
- **Drafts community replies** in Iksula's approved "voice" — but a human must approve any reply that posts under a named leader's name.
- **Captures who engaged** — every comment, reply, DM (direct message), form fill, content download, tracked link click, connection-accept, or mention.
- **Hands interested people to Lead Gen** — one record per interested person, raw and unjudged.

What it **does NOT do** (these belong to other agents):

- It does **not qualify** anyone — no scoring, no "hot/cold," no labeling someone a good lead. It passes interest along **verbatim** (exactly as observed).

- It does **not** figure out who an anonymous visitor really is, look up their company, or merge duplicates. That is Lead Gen's job.
- It does **not** do one-to-one outreach — no personal DMs or emails to a named prospect. (Same platforms as Lead Gen, but the opposite motion: broadcast vs. targeted.)
- It does **not** create the content, choose the channels, or set the budget — those are earlier agents.
- It does **not** own the numbers/analytics — it feeds raw figures upward and lets the Brain own the analysis.

The iKshana assembly line — and where this agent sits

Iksula is building **iKshana**, an AI-native marketing-and-sales organisation: a chain of AI agents, each doing one job and passing its work to the next, like stations on a factory line. The order is:

Thought Leadership → Media Planner → Content Creator → Media Planner → Growth Hacker → Lead Gen → Sales/KAM

In plain terms:

Station	Plain-English job
Thought Leadership	Decides the big ideas and themes to be known for
Media Planner	Picks the channels, schedule, and budget
Content Creator	Actually writes/produces the posts and assets
Growth Hacker (this agent)	**Publishes it all, runs the tests, captures who engaged**
Lead Gen	Qualifies the interested people and prepares outreach
Sales / KAM	Closes deals (KAM = Key Account Management, for existing clients)

The skill calls the Growth Hacker **"the last fast-loop Hand"** — the final step that turns a plan into live action, and the first step of the feedback that flows back to improve future plans. It is **fed by** the Content Creator (the finished content) and the Media Planner (the channel plan and budget), and it **feeds** two things forward: the **Brain** (raw performance numbers) and **Lead Gen** (interested people).

Brain, Spine, Hands, and Zoho — a kitchen analogy

iKshana is organised into four parts. Picture a busy restaurant kitchen:

- **The Brain** is the **recipe book and the tally sheet on the wall** — a Google Drive folder called `_brain/`. It holds shared knowledge and aggregate numbers (totals, averages). It is **append-only** (you can add pages, never erase or overwrite) and contains **no personal data** about any individual. The Growth Hacker is allowed to write to exactly **two pages** here: `performance-analytics-growth-YYMMDD` (how each post performed, as numbers) and `channel-intel-growth-YYMMDD` (what each channel rewarded this cycle, plus which experiments won).
- **The Spine** is the **order rail and the ticket spike** — a Drive folder called `_spine/`. It holds the live to-do queues and coordination records, and **can change** as work moves along. The Growth Hacker reads its orders here (the publishing calendar, the media plan, the content bundle) and posts back its progress (the execution log, experiment results, and the new-lead tickets).
- **The Hands** are the **cooks** — the agents that actually do things. The Growth Hacker is one Hand.
- **Zoho CRM** is the **reservations book with each guest's name and history** — the system-of-record for per-person lead data (the live, changeable, personal details). Notice the Growth Hacker does **not** write to Zoho. Personal lead data is deliberately kept out of the Brain (which has no personal data and can't be edited) and handled downstream by Lead Gen via Zoho.

Each agent is two things working together: a **SKILL** (the markdown instructions the AI reads and follows) and a **TOOLKIT** (Python code that *enforces* the safety rules, so the agent literally cannot do the dangerous thing even if it tried). For the Growth Hacker, that toolkit lives at `tools/growthhacker/` and includes a publish gate, the seam emitter, and the Brain writer.

"The seam" — the one hand-off to Lead Gen

The **seam** is the single, one-way doorway between the Growth Hacker and Lead Gen. The skill is strict about this: **one seam, one direction (Growth Hacker → Lead Gen), one object** — a record called `content-sourced-lead`.

Here is the plain-English deal. Whenever someone shows interest in a post (comments, replies, fills a form, clicks a tracked link, etc.), the Growth Hacker drops **one ticket** onto a queue in the Spine that Lead Gen picks up. That ticket carries:

- A unique ID (a **ULID** — a one-of-a-kind code) so that if the same ticket is dropped twice, it counts only once.
- What was actually observed about the person (a handle, a profile link, or an email only if they volunteered it) — **never** dressed up or looked up.
- Which post it came from, which channel, and the **exact** action they took.
- A **region** and a **lawful-basis tag** — a stamp marking what the law allows. US → a person may be identified later; EU / India / anywhere else → **company-level only** (because of privacy laws like GDPR and India's DPDP). The toolkit fails safe: anything not clearly US is stamped company-level.

Crucially, the ticket carries **no score and no "is-this-a-good-lead" verdict**. The Growth Hacker surfaces the raw signal and stops there. Everything after — figuring out who the person is, looking up their company, removing duplicates, getting lawful-basis sign-off, deciding new-customer-vs-existing-client routing, and qualifying the lead — is **owned once, by Lead Gen**. The toolkit's seam emitter even *rejects* any score, grade, or status field if you try to sneak one in, and it writes only to the Spine queue, never to the Brain. (Aggregate counts — "how many content-sourced leads this week, by channel" — do flow to the Brain, but only as numbers, never as the personal lead rows themselves.)

One honest note about today (June 2026)

This agent is **built and safety-verified, but it is not publishing or sending live yet.** The toolkit's own README says plainly that the live distribution connectors (LinkedIn / Telegram / X) are **not wired up yet**, and publishing is held behind a kill-switch (an on/off master switch) called `GROWTH_PUBLISH` that defaults to off. So today the Growth Hacker can plan, gate, and dry-run safely — but the moment it actually posts to the public is still ahead, switched on deliberately by a human.

2 How it was made

The Growth Hacker agent is built from two parts that work together. Think of them as **a recipe** and **a kitchen that has been child-proofed**. One tells the AI what to do; the other physically stops it from doing anything dangerous, even if it wanted to.

Part 1 — The SKILL (the plain-English instructions)

A "skill" is just a long, carefully-written instructions document in plain English (technically Markdown, a simple text format). The AI reads it before it does any work, and follows it like a checklist.

The Growth Hacker's skill lives in one main file plus three reference notes:

- `SKILL.md` — the master instructions: what the agent is for (publish the already-approved content calendar to LinkedIn, X, YouTube, blog, Telegram), what it is allowed to touch, what it must never touch (it must never "qualify" a lead or score anyone — that is Lead Gen's job), and the step-by-step workflow from "read the plan" to "capture who engaged".
- `references/distribution-playbook.md` — the channel-by-channel how-to (best posting times, how to reshape a post for each platform).
- `references/experiment-design.md` — the rules for running A/B tests (small experiments where you try two versions and keep the winner).
- `references/seam-contract.md` — the exact rules for handing an interested person over to the Lead Gen agent.

The skill is written in a "surface, don't decide; execute, don't re-plan" style. In plain terms: the Growth Hacker does the work, but it never overrules the plan and never makes the big judgement calls. If something looks wrong, it raises a flag instead of improvising.

Part 2 — The TOOLKIT (small Python programs that enforce the rules)

The problem with instructions alone is that instructions can be misread or ignored. So the dangerous actions are not left to the AI's good behaviour — they are handled by small, fixed computer programs (written in Python, a common programming language) that sit under `tools/growthhacker/`. The AI is told to **call these programs rather than do the risky step itself**.

The programs simply refuse to do the wrong thing.

Here is what each program does, in plain language:

Program (file)	What it enforces
`publish_gate.py`	The bouncer at the door. Before anything is published it checks: is the master "GROWTH_PUBLISH" switch on? Is this an approved, scheduled item? Does it have tracking attached? Did a human approve it where required? If any check fails, it returns **HOLD** instead of ALLOW.
`seam_emitter.py`	The clean hand-off to Lead Gen. It writes one tidy record per interested person and **strips out anything it must not carry** (any score, grade, "MQL/SQL" label, or account status).
`brain_metrics.py`	The note-taker for the shared knowledge folder. It only allows **aggregate numbers** (like "47 comments this week") and **rejects any row that looks like a real person's email, name, or handle.**
`config.py`	The rulebook the others read — the allowed channels, the forbidden words, the on/off switch.
`preflight.py`	A "dry run" tester. You can rehearse a publish or a hand-off and it tells you ALLOW or HOLD **without anything actually going out.**

"Safe by construction" — the no-send-button idea

The phrase **"safe by construction"** means the safety isn't a warning sticker you could peel off — it's built into the shape of the thing so the bad action is impossible.

A simple analogy: imagine a microwave with **no door-open-while-running wiring at all.** You can't get burned by opening it mid-cycle, because the engineers never connected that path. It's not that someone remembered to be careful — the danger simply doesn't exist in the machine.

That is exactly how this toolkit is built. A few concrete examples from the actual code:

- **There is no "send email" or "post to LinkedIn" code in this package at all.** The README says it plainly: *"nothing here posts publicly or sends email."* The publish program (`publish_gate.py`) only ever returns the word **ALLOW** or **HOLD** — it is a decision, never an action. So the AI literally cannot press a "send" button, because no such button was ever wired up.
- **A master kill-switch defaults to OFF.** In `config.py`, the `publish_enabled()` function reads an environment setting called `GROWTH_PUBLISH`. Unless it is explicitly set to `on`, **everything is held.** Today it is off, so the correct number of posts going out is zero — by design, not by luck.
- **The hand-off to Lead Gen cannot smuggle a "score".** Sending someone a sales score is forbidden for this agent. The `seam_emitter.py` program scans the whole record — *even inside nested sub-fields* — for any banned word like `score`, `grade`, `mql`, `tier`, or `rating`, and **flatly refuses** (raises an error) if it finds one. So even an AI that "tried" to attach a score would be blocked by the program.
- **No personal data can leak into the shared knowledge folder.** The Brain (the append-only knowledge folder) is for aggregate numbers only. `brain_metrics.py` checks every row against a fixed list of columns and uses pattern-matching to spot anything that looks like an email address or an `@handle`, and rejects it. A real person's details simply cannot be written there.
- **Non-US contacts are protected automatically.** The `lawful_basis_tag()` function is **fail-safe**: anything not clearly marked "US" is automatically tagged "company-level only (GDPR/DPDP)" — the stricter, safer setting. The safe default is the automatic one.

Stress-tested by independent reviewers — twice

This wasn't just self-graded. The toolkit ships with **28 automated safety tests** (small programs that try to trick it and confirm it refuses). They run with one command from the `tools/` folder:

```
python -m unittest growthhacker.tests.test_safety
```

On top of that, the package was put through **adversarial "red-team" review** — independent AI reviewers whose job is to attack the design and find a way to make it misbehave (post something, leak data, sneak a score across). Both the SKILL and the toolkit README record the verdict: the safety design was found **SOUND** and the tests all pass. The toolkit deliberately copies the same proven pattern already used by the Lead Gen toolkit (`leadgen/`), which went through the same

"safe-by-construction, adversarially verified" treatment.

Honest note on what is NOT built yet

To be straight with you, the README is candid about the current limits — this is real but unfinished:

- **The live posting connectors are not wired in yet.** There is a manual `telegram_publish.sh` helper, but connecting LinkedIn / X / Telegram *through* the gate is described as "the next slice." So today the agent prepares and decides, but a human still does the actual posting.
- **The Brain and Spine storage are local stand-ins for now.** The programs currently write to a local `_state/` folder; the real Google Drive `_spine/` and `_brain/` folders get wired in later.
- **Matching a reply to the right "voice" persona is still a follow-up.** Today the safeguard is simply: a named-leader reply needs a human's approval token before it can post.

In short: the recipe (SKILL) and the child-proofed kitchen (toolkit) are both built and independently verified safe. The stove just hasn't been turned on yet — and turning it on is a deliberate, human-controlled step, not something the agent can do on its own.

Proof to point to: the toolkit README at `tools/growthhacker/README.md`, and the two modules that do the heavy lifting — `tools/growthhacker/publish_gate.py` (the bouncer) and `tools/growthhacker/seam_emitter.py` (the clean hand-off).

3 Everything it can do — with real examples

Think of the Growth Hacker as Iksula's **distribution operator**. Someone else (the Content Creator) writes the posts. Someone else (the Media Planner) decides which channels and how much money. The Growth Hacker's whole job is to **take that already-approved plan and make it happen out in the world** — put the posts up, run the small tests the plan asked for, talk in the comments, watch who responds, and pass interested people along to the Lead Gen agent.

A useful rule to remember as you read: the Growth Hacker **executes the plan, it never re-plans**. If something looks wrong, it raises a flag — it does not quietly fix the strategy itself.

Below is everything it can do, grouped into the eight things it actually delivers. For each one you get a plain description, the kind of instruction you'd type, and what you'd get back.

A quick note on two terms used throughout:

- **Brain** = a Google Drive folder (`_brain/`) holding shared knowledge and aggregate numbers — no personal data, write-once (you can add, never edit).
- **Spine** = a Drive folder (`_spine/`) holding the shared to-do lists and run records — these *can* change. The publishing calendar, the media plan, and the queue of leads all live here.

1. Read the plan and check it before touching anything (Intake & grounding)

What it does: Before it publishes one thing, the agent opens the **publishing calendar** (the list of what posts where and when), the **media plan + budget** (channels, schedule, any spend, and whether spend was approved), and the **asset bundle** (the actual finished content). It then pulls four reference files from the Brain: `channel-intel` (what each platform rewards right now), `icp-audience` (who the audience is, plus the **geography map** — US vs EU/India/other), `competitor-radar` (timing gaps to aim for), and `voices` (the approved personalities for named-leader replies). Then it sanity-checks every calendar row.

The check is strict. If a calendar row points to a post that doesn't exist, or a dependency isn't met, or a *paid* row has no budget approval — the agent **STOPS and flags it**. It does not improvise.

You'd type:

"Run the growth hacker — intake this cycle's calendar and tell me if anything's missing before we publish."

You'd get back: a readiness report, e.g. *"7 of 8 calendar rows resolve to a real asset. Row 4 (LinkedIn carousel) points to* `asset_ref ART-0412` *which I can't find in the asset bundle, and Row 6 is a* paid *X post with no budget-approval flag from the Media Planner. Both are HELD — I've posted a flag to* `#agentic-org-requests`*. The other 6 rows are clear to*

schedule."

2. Schedule each post for the right moment (Schedule)

What it does: For every approved row, it nails down the exact send time per channel and timezone — using the plan's posting windows, what `channel-intel` says works this quarter, and `competitor-radar` for **timing whitespace** (posting into the quiet gaps rivals leave). It then advances the row's status from **Planned → Scheduled**. It owns the *clock*, not the calendar — it never changes who the audience is, the message, or the funnel intent.

You'd type:

"Schedule this cycle's posts — find the best slots and avoid the windows competitors are crowding."

You'd get back: e.g. *"LinkedIn company post scheduled for Tue 09:10 IST (business-day morning window; competitors cluster at 11:00, so I moved earlier into the gap). X thread scheduled Wed 18:30 IST. Blog edition Thu 08:00. All 6 rows now status = Scheduled."*

3. Reshape each post to fit the platform — never rewrite it (Reshape)

What it does: A long article isn't a tweet; a tweet isn't a YouTube title. The agent adapts each asset to its platform's **native form** — length, aspect ratio, thread-vs-carousel, where the hook sits. The hard line: it can change *shape and formatting*, but it may **never** change the claim, the proof, the thesis, the target audience, or the message. Those belong to the Content Creator and the plan. If a post can't go native without changing its argument, it flags it instead of fixing it.

Here's the native shape it produces per surface:

Channel	What it reshapes to
LinkedIn company page	single image / document carousel / text+link; hook in the first 1–2 lines (to survive the "see more" cut)
LinkedIn profile (a named leader)	first-person POV post or short thread
X / Threads	thread vs single; tightened to platform length; hook first
Blog / owned site	full edition + on-page CTA / form
YouTube / short video	storyline beats re-cut to aspect ratio + length; title = a 5-second hook
Newsletter / owned email	issue layout; subject line = the hook

You'd type:

"Reshape asset ART-0410 (the 'returns-fraud' article) for LinkedIn and X."

You'd get back: a LinkedIn carousel draft with the punchline pulled into line 1, plus a 5-tweet X thread version — both carrying the **same** argument and claims as the original article, just re-shaped. If the argument couldn't survive the cut, you'd instead get: *"Flagging — this can't go native on X without dropping the core proof point; sending back to the Spine."*

4. Instrument every post so you can measure it (Instrument before publish)

What it does: Nothing ships "blind." Before any post goes out, the agent attaches **UTM tags** (little tracking codes on a link that tell you which post a click came from), **tracked CTAs** (call-to-action buttons/links you can count), and per-asset measurement — so every click and form-fill traces back to the exact post and test variant. This is enforced in code: the publish gate **HOLDS** any organic, paid, or broadcast-email action that arrives without both a UTM and a tracked CTA.

You'd type:

"Instrument the LinkedIn and blog posts for this cycle before they go out."

You'd get back: each post stamped with a UTM string tying it to its `asset_ref` and variant, and the blog form wired for capture — confirmed as "instrumented" so the gate will let it through.

5. Publish and distribute — with a hard gate on paid (Publish + distribute)

What it does: It runs the actual distribution per the calendar, advances each row **Scheduled → Published**, and writes an **execution log** to the Spine as it goes (what shipped, when, on which channel, and any deviation). Every publish decision runs through a piece of code called the **publish gate** (`growthhacker.publish_gate.evaluate()`) that returns **ALLOW** or **HOLD**. The agent literally cannot post if the gate says HOLD.

The gate enforces four rules, and a master kill-switch sits above all of them:

- **Kill-switch:** an environment setting named `GROWTH_PUBLISH` must be set to `on`. If it's anything else, **everything is HELD**. This is off today.
- **Organic post:** no separate human sign-off (the calendar was already approved), but it **must** be instrumented and executing a **Scheduled** row.
- **Paid post: HELD unless `budget_approved` is true** — inherited from the Media Planner. The Growth Hacker never approves spend itself.
- **Named-leader reply: HELD unless a human approval token** is present (see #7).
- **Broadcast email: HELD unless a human approves the first send** (see below).

Honest status today: nothing actually posts. The kill-switch is off, and the live connectors to LinkedIn / X / Telegram aren't wired through the gate yet. A `telegram_publish.sh` helper exists as a manual tool, but wiring it through the gate is still on the to-do list. So today this step **decides and drafts** — it does not push live.

You'd type:

"Publish this cycle's scheduled organic posts."

You'd get back (today): *"GROWTH_PUBLISH is off, so all publishes are HELD by design — nothing went live. Here is the ALLOW/HOLD verdict for each row so you can see what* would *ship once the switch is on, plus the execution-log entries staged in the Spine."*

You can also preview the gate's verdict on any single action without posting anything, using the dry-run tool:

```
python -m growthhacker.preflight --action action.json
```

which prints `=== PUBLISH: ALLOW ===` or `HOLD` with the exact reasons (e.g. *"Held for: paid_without_budget_approval (Media Planner gate); not_instrumented"*).

6. Run small A/B experiments — strictly within the plan's bounds (Run experiments)

What it does: It tests *how* an approved post is distributed — never *what* it says. An **A/B test** means showing two versions (A and B) and seeing which performs better on one chosen measure.

What it **may** test vs **may not**:

MAY test (distribution)	MAY NOT test (someone else owns it)
Hooks — the first line / headline framing of the *same* post	The thesis, argument, or claims (Content Creator)
Post times — within the plan's window	The channel mix or budget split (Media Planner)
Audience cuts the plan already allows	Who the target ICP is (the plan)
CTA wording / placement	Funnel intent (top/middle/bottom-of-funnel)
Native format — thread vs carousel, length	The cadence / sequencing (the Spine)

The method is disciplined to avoid fooling yourself. Before launch it **pre-registers**: one falsifiable hypothesis, the two variants (one variable at a time), **one** primary metric chosen up front (no switching metrics after the fact), an **audience-type control** (it must never mix "new" and "retargeting" audiences in one comparison — that's the #1 way to get a fake result), and a sample size + stop-rule. Then **no peeking** — it declares a winner *only* when the stop-rule is hit. An out-of-bounds idea becomes a Spine flag, not an experiment.

You'd type:

"Set up an A/B test on the carousel ART-0410: question-hook vs stat-hook, primary metric saves, new audience only, stop at 2,000 impressions per arm."

You'd get back: a pre-registered experiment card — hypothesis ("a question-hook lifts saves vs a stat-hook for the same carousel, new audience"), the two variants, primary metric = saves, audience_type = new (held constant), sample = 2,000/arm, no-peek stop-rule. Later, when the rule is met, you get a **winner + lift** (e.g. *"Variant A, question-hook, +38% saves, declared at the stop-rule"*), written to the Spine **experiment-results** record to inform the *next* cadence — the Spine, not the agent, decides cadence.

7. Draft and post community engagement — voice-gated (Community engagement)

What it does: It does the 1-to-many community work: monitoring threads, replying to comments, proactively commenting, native to each platform. The critical guardrail: any reply posted **as a named leader** (e.g. "as DJ") is **drafted against that person's persona in the Brain** `voices` **file** — and a **human must own and approve** that voice before it posts. The skill **drafts, never invents** a voice. In code, the gate **HOLDS** any `community_reply_as_byline` action unless a human `approval_token` is attached — and that gate is always on for byline replies; you can't switch it off with a flag.

Hard line to remember: a **public reply on a post is Growth Hacker**. A **private, personalised DM to a named prospect is NOT** — that's Lead Gen's 1-to-1 motion. The agent also avoids generic "Great insights!" bot-filler and never publicly disparages competitors.

You'd type:

"Draft replies as DJ to the three comments on yesterday's LinkedIn carousel."

You'd get back: three reply drafts, each grounded in the actual comment text and written in DJ's `voices` persona — clearly marked **HELD pending human voice approval**. Once a human approves (supplies the token), the gate flips to ALLOW. Without the token, they stay drafts.

It can also draft **company-voice** replies for the company account (same draft-then-approve discipline for sensitive threads), and offer **employee-advocacy reshare variants** — pre-approved, opt-in-only versions for staff to reshare (no auto-posting from someone's account, no identical-copy mass reshare that looks like a bot).

8. Detect interest and hand it to Lead Gen — the seam (Detect + emit)

What it does: This is the agent's most important handoff. Whenever someone engages — a comment, a reply, a DM on a public post, a form fill, a content download, a tracked CTA click, a connection-accept, or a mention — the agent writes **one** `content-sourced-lead` record into a queue in the Spine that Lead Gen reads. It does this through code (`growthhacker.seam_emitter.emit()`), never by hand.

What the record carries, and the strict limits:

- A **unique ID** (`correlation_id`, a ULID) so re-sending the same event does nothing (no duplicates).
- `contact_identity` — only what was *actually observed*, verbatim and **unenriched** (a handle, a profile URL, or an email *only if the person volunteered it*). It may be partial or anonymous.
- `source_asset_ref` and `source_channel` — which post and surface it came from.
- `intent_signal` — the raw action, verbatim, with a timestamp. **No score.**
- `region` **+** `lawful_basis_tag` — stamped from the geography map. The rule is **fail-safe**: US → "person-level de-anon permitted"; **anything not clearly US (EU/India/other/unknown) → "company-level ONLY (GDPR/DPDP)"**, the more restrictive setting. The agent stamps this; Lead Gen *enforces* it.

What it is **forbidden** to do, enforced in code: it must **never** put a score, grade, MQL/SQL label, qualification, enrichment, account-status, or ack on the record. The emitter scans the whole record **recursively** — a score hidden inside a nested field is rejected too — and leaves `ack_status` empty for Lead Gen to fill in. It also never de-anonymizes a visitor, never de-dupes into a CRM, and never routes by account status. Those are all Lead Gen's job. And Reference-channel leads never come through this seam.

You'd type:

"Capture this cycle's engagement and emit the content-sourced leads to Lead Gen."

You'd get back: e.g. *"Emitted 14 content-sourced-lead records to the Spine queue. Example: a LinkedIn comment from an EU profile → stamped* `non-US: company-level ONLY (GDPR/DPDP)`*, no score, ack_status left null. 2 duplicate re-detections were no-ops (same ULID). 0 records carried a forbidden field."* If you tried to slip a score in, you'd see it

rejected: *"seam record must not carry ['lead_score'] (GH never qualifies/acks/enriches)."*

You can dry-run a single event safely (it writes to a throwaway queue, never the real Spine):

```
python -m growthhacker.preflight --event event.json
```

9. Return raw performance to the Brain — aggregates only (Capture + return)

What it does: It rolls up engagement by **asset × format × channel × time** and writes it back to the Brain through a guarded writer (`growthhacker.brain_metrics.write_growth_metrics()`). Two feeds:

- `performance-analytics-growth-YYMMDD` — the raw engagement numbers, in a fixed schema: `period | source | asset_or_campaign | channel | metric | value | audience_type | notes`.
- `channel-intel-growth-YYMMDD` — the synthesis: what each channel rewarded this cycle + experiment winners.

The metrics it owns are engagement/distribution ones: impressions, reach, reactions, comments, shares, saves, clicks, views, watch-time, email opens/CTOR (for email it broadcasts), reshare lift, experiment lift. The `audience_type` (new vs retargeting) is **always stamped**, so numbers are read honestly. Two hard limits, enforced in code: **aggregates only, never per-person PII** (the writer rejects any row that looks like an email, a handle, a name, or a nested object), and it **never reports funnel conversion** (MQL→SQL→meeting) — that's Lead Gen's separate feed. **Raw-first**: it dumps the raw export to `_brain/_raw/` before publishing any synthesised number, so every figure is traceable. Counts of content-sourced leads (how many, by channel) flow here as **numbers** — never the lead rows themselves (those stay in the Spine queue).

You'd type:

"Capture and return this cycle's performance to the Brain."

You'd get back: confirmation that, say, 22 aggregate rows were written to `performance-analytics-growth-260623` (raw dumped first), plus a `channel-intel-growth-260623` note like *"LinkedIn carousels outperformed text posts 3:1 on saves this cycle; question-hooks won the A/B; new-audience CTOR on the newsletter was 4.1%."* If a row accidentally contained an email address, you'd see it **rejected** with *"value looks like PII (email/@handle) — the Brain is aggregates-only."*

10. Flag problems instead of silently fixing them (throughout)

What it does: Across every step, when the agent hits a missing asset, an unmet dependency, a paid row with no approval, a channel anomaly (e.g. an account acting throttled), or a request that isn't actually its job — it **surfaces a flag** to the Spine / `#agentic-org-requests` and **routes the request** to the right owner (Lead Gen, Media Planner, Content Creator, or the Spine) rather than absorbing scope it shouldn't have.

You'd type:

"Run lead qualification on these new leads and email them."

You'd get back: a polite decline + route — *"Qualification, enrichment, de-dupe and any 1-to-1 email are Lead Gen's, not mine. I've handed these to the Lead Gen seam unqualified and verbatim. I don't score or send."*

The bottom line on what's live today

Built, tested (28 safety tests, "adversarially verified SOUND"), and ready — but deliberately **not posting or sending live yet**:

- The **kill-switch** `GROWTH_PUBLISH` **is off**, so every publish is HELD.
- Live connectors (LinkedIn / X / Telegram) aren't wired through the gate yet.
- The Spine/Brain writes currently go to **local mirror files** under `_state/`; production will route them to real Drive `_spine/` + `_brain/` via `brain_io`.

So today the Growth Hacker will happily **plan, schedule, reshape, instrument, draft, decide ALLOW/HOLD, and stage seam records and metrics** — and it will tell you plainly that nothing actually went public, exactly as designed until a human turns it on.

4 The tools and integrations it is connected to

Think of the Growth Hacker as a careful worker who never touches a dangerous machine directly. Everything it does flows through a small set of named programs and connectors. Some of these are LIVE (working today). Several of the "outside world" connectors — the bits that would actually post to LinkedIn or send an email — are deliberately NOT built or are switched OFF. That is on purpose, not an oversight.

Below is the complete list, split into two groups: (1) the agent's own toolkit (small Python programs that ENFORCE the safety rules — "Python" just means the programming language they are written in), and (2) the outside tools, apps and connectors it touches.

Group 1 — The agent's own toolkit (the safety rails)

These live in one folder inside the plugin: `tools/growthhacker/`. The agent is told to CALL these programs rather than improvise — they make the safety rules structural, meaning the agent literally cannot do the forbidden thing because the program refuses. None of them post in public or send any email. All are LIVE today.

Toolkit program	What it does	How the agent uses it	Status
`publish_gate.py`	A traffic light for every publish action. It returns either ALLOW or HOLD.	Before anything "ships", the agent runs `publish_gate.evaluate()`. It HOLDs unless the action passes every check (see the four gates below).	LIVE (it only decides; it never actually posts)
`seam_emitter.py`	Writes ONE "content-sourced-lead" record — the handoff to the Lead Gen agent — into the shared to-do queue.	The agent calls `seam_emitter.emit()` for each interest event it spots (a comment, a form fill, a click).	LIVE (writes to a LOCAL file mirror today, not the real shared Drive — see "transport" below)
`brain_metrics.py`	The only approved way to write aggregate numbers (totals, never names) into the Brain.	The agent calls `write_growth_metrics()` to record engagement counts by post, format, channel and time.	LIVE (writes to a local mirror today)
`preflight.py`	A "dry run" — test a planned post and a planned handoff WITHOUT doing anything for real.	The operator runs `python -m growthhacker.preflight --action action.json --event event.json`. The seam test is written to a temporary throwaway file, so the real queue is never touched.	LIVE
`ulid.py`	Makes the unique ID stamped on each handoff record (a ULID is just a sortable, one-of-a-kind reference number).	Used automatically by `seam_emitter.py` so re-sending the same event twice is harmless (a "no-op").	LIVE
`audit.py`	A provided logging helper that scrubs out private details (PII) and secrets before writing a line. **Note:** in this version the publish-gate and seam-emitter do not yet route through it — wiring audit into every action is part of the not-yet-built live path.	append-only, metadata only	Helper (not yet wired in)
`config.py`	The rulebook of hard constants: the allowed channels, the four valid action types, the banned words on the handoff, and the kill-switch.	Read by all the programs above; not run directly.	LIVE

The four gates `publish_gate.py` enforces (the agent cannot bypass any of them):

1. Paid post → must carry the Media Planner's budget approval (`budget_approved`), else HOLD.
2. A reply posted as a named leader → must carry a human `approval_token` (a human owns that person's voice), else HOLD.

3. A broadcast/BOFU email the Growth Hacker sends itself → must carry a human first-send `approval_token`, else HOLD.
4. Any post at all → must be "instrumented" (carry a UTM tag and a tracked link — UTM is the little tracking tag added to a web link so you can see where a click came from) AND must be running an already-approved calendar row whose status is exactly "Scheduled". Organic posts have NO separate human gate (the plan was already approved), but they still must pass this check.

The master kill-switch: an environment variable (a setting read from the operating system) named `GROWTH_PUBLISH`. Unless it is set to `on`, EVERY publish is held. Today it is effectively off — so the correct number of live posts is zero until a human turns it on.

Group 2 — The outside tools, apps and connectors

Tool / connector	What it is	What the agent uses it for	How it connects	LIVE or PENDING today
Google Drive `_spine/` (the Spine)	A Drive folder holding the shared queues and run-records that agents read/write.	Reads the publishing calendar, the media plan + budget, and the asset bundle. Writes the execution log, experiment results, and the content-sourced-lead handoff queue that Lead Gen polls.	Via `brain_io` (the shared Drive helper). The Drive connector is reached through Google Drive MCP tools (MCP = a standard "plug" that lets the AI call an outside service).	PENDING for real Drive transport. Today the toolkit writes a LOCAL mirror under a `_state/` folder; production will resolve the real Drive `_spine/` via `brain_io`.
Google Drive `_brain/` (the Brain)	An append-only Drive folder of knowledge and aggregate numbers — NO personal data.	Reads `channel-intel`, `icp-audience` (incl. the geography map), `competitor-radar`, and `voices`. Writes two aggregate feeds: `performance-analytics-growth-YYMMDD` and `channel-intel-growth-YYMMDD`.	Via `brain_io`. Note the rule: read feeds with `download_file_content`, NEVER `read_file_content` (the latter corrupts the spreadsheet data).	PENDING for real Drive transport (local mirror today, same as the Spine).
`brain_io`	The shared helper that resolves Drive folder IDs at runtime and reads/writes Brain and Spine files.	The single approved doorway to Drive — the agent never hardcodes folder IDs.	Plugin helper that sits on top of the Google Drive connector.	Helper is defined; live Drive wiring is PENDING (see above).
Slack `#agentic-org-requests`	A channel in the Iksula Services Slack workspace used as the human approval / flag line.	Where the agent posts plan/asset problems (missing asset, unmet dependency, channel anomaly) and where human gate approvals are raised.	Slack. (Per project history the bot posts directly via Slack's message API; approval is a human reply.)	LIVE as the coordination channel; the agent's own auto-posting to it is part of the not-yet-wired live path.
LinkedIn (company page + leader profiles)	The main 1-to-many publishing surface. Allowed surfaces in the rulebook: `LinkedIn-post`, `LinkedIn-page`.	Would publish posts/carousels/threads and capture engagement; community replies as a named leader are voice-gated.	A live publishing connector behind `publish_gate`.	PENDING — explicitly listed as "intentionally NOT built yet." No live connector exists.
X / Threads	A 1-to-many publishing surface. Allowed: `X-post`, `X-thread`.	Would publish threads/single posts and capture engagement.	Live connector behind `publish_gate`.	PENDING — not built yet.

Tool / connector	What it is	What the agent uses it for	How it connects	LIVE or PENDING today
Telegram	A 1-to-many channel. Allowed: `Telegram`.	Would publish to a Telegram channel.	A manual helper script `telegram_publish.sh` exists; wiring it THROUGH `publish_gate` is named as "the next slice."	PARTIAL/PENDING — manual helper only; not yet gated or live.
YouTube / short-video	A publishing surface. Allowed: `YouTube-video`, `YouTube-comment`.	Would publish videos and capture views/saves; comments are community engagement.	Live connector behind `publish_gate`.	PENDING — not built yet.
Blog / owned website	The owned-site surface. Allowed: `Blog-post`, `Blog-form`.	Would publish the blog edition with an on-page form/CTA and capture form fills + clicks.	Live connector / site publishing behind `publish_gate`.	PENDING — not built yet.
Owned email / newsletter (broadcast)	A broadcast email the Growth Hacker might send itself (NOT 1-to-1 outreach — that is Lead Gen's job).	Would send a newsletter issue and capture opens/CTOR.	Behind `publish_gate`; the FIRST send of any new sequence is human-gated.	PENDING / OFF — no send happens; first-send is human-gated and the kill-switch is off.
UTM + tracked-CTA instrumentation	The tracking tags and tracked links attached to every post so engagement can be tied back to the exact asset.	Required by `publish_gate` before any organic/paid/email ships; the surface `tracked-CTA-click` is also a valid interest signal.	Built into the content at publish time.	The RULE is LIVE and enforced; the live publishing that would apply it is PENDING.
Zoho CRM	Iksula's system-of-record for per-person lead data.	NOT used by the Growth Hacker. It deliberately never touches the CRM — it hands raw interest to Lead Gen, and Lead Gen owns all CRM work (de-dupe, enrichment, lawful-basis sign-off).	Zoho CRM connector — used downstream by Lead Gen, not here.	Out of scope for this agent by design.
Lead Gen agent (downstream)	The next agent in the chain.	Receives every `content-sourced-lead` the Growth Hacker emits, verbatim and unqualified, and does everything after.	Via the Spine queue (the handoff file), not a direct call.	The handoff WRITE is LIVE in mirror form; the live polling is operator/convention-based in the pilot.

Two important plain-English notes

- **The handoff carries no judgement.** When the Growth Hacker hands an interested person to Lead Gen, the record is stamped with a region and a "lawful basis tag" (a label saying what is legally allowed). The rule is fail-safe: anything that is not clearly US is marked "company-level ONLY" (no person-level chasing) to respect EU GDPR and India DPDP privacy law. The toolkit also REJECTS any attempt to attach a score, grade, MQL/SQL label or account-status — even if someone tries to hide one inside a nested field. Scoring and qualifying are Lead Gen's job alone.
- **Nothing here goes public today.** The toolkit's own README states it plainly: "nothing here posts publicly or sends email." The live distribution connectors (LinkedIn, X, Telegram, YouTube, blog, email) are listed as not built yet, the real Drive transport is a local mirror for now, and the `GROWTH_PUBLISH` kill-switch must be deliberately turned on before a single post can ship. So today the Growth Hacker can plan, dry-run, gate-check and write mirror records — but it does not actually publish to any platform.

5 How to set it up — step by step

This is the plain-English checklist to get the Growth Hacker (the agent that distributes Iksula's already-approved content and hands interested people to Lead Gen) ready to run. Follow it in order. Nothing here makes the agent post in public or send email — that stays switched OFF until the very end, on purpose.

A quick mental model before we start. The agent is two things bolted together:

- A **skill** — a set of plain-English instructions the AI reads and follows. For the Growth Hacker, that file is `skills/growth-hacker/SKILL.md`.
- A **toolkit** — actual computer code (under `tools/growthhacker/`) that *enforces* the safety rules, so the AI literally cannot do the dangerous thing even if it tried. This is the part you install and test below.

Step 1 — Get the accounts you need (and know why)

You log in as one company identity and connect a handful of services. Here is the full list and the reason for each:

Account	What it is	Why the Growth Hacker needs it
@iksula.com Google Workspace	Iksula's company Google account (email + Google Drive)	This is the identity that owns the **Brain** and **Spine** folders in Google Drive. The agent reads its plan and writes its results here. Use this account, not a personal Gmail.
Zoho CRM	The system-of-record (the master list) for per-person lead data	The agent itself does **not** write leads into Zoho — Lead Gen does. But Zoho is the home of the real lead data, so it is connected once, the safe way (Step 4).
Woodpecker	The email-sending platform	Same note: the Growth Hacker does not send email here. This is a **live account with 97 real campaigns**, which is exactly why everything is locked down.
Slack	Team chat	The agent posts problems and approval requests to the `#agentic-org-requests` channel (in the "Iksula Services" workspace). When the agent hits a missing asset or needs a human to approve a voice, that is where it speaks up.
Apollo	A contact/intent database	Used downstream by Lead Gen, not by the Growth Hacker directly. Listed here only so you have one complete account list for the whole assembly line.

Plain takeaway: the **only** account the Growth Hacker truly leans on day-to-day is the **@iksula.com Google Drive** (for the Brain and Spine) plus **Slack** (for flagging). The rest are part of the wider system.

Step 2 — Install the plugin and skill

The Growth Hacker lives inside the "iksula-agents" plugin. Installing the plugin makes both the skill (the instructions) and the toolkit (the safety code) available.

- The skill file is at: `iksula-marketplace/plugins/iksula-agents/skills/growth-hacker/SKILL.md`
- The toolkit (Python code) is its sibling at: `iksula-marketplace/plugins/iksula-agents/tools/growthhacker/`

"Python" just means the toolkit is written in the Python programming language — you do not need to write any; you only run a couple of ready-made commands later to test it. Once the plugin is installed in Claude Code, you trigger the agent by typing a phrase like **"run the growth hacker"**, **"publish the calendar"**, or **"distribute this cycle's content"**.

Step 3 — Connect and verify the Brain/Drive (the "pranaam" startup skill)

The **Brain** is a Google Drive folder called `_brain/` (append-only knowledge and aggregate numbers — never any personal data). The **Spine** is a Drive folder called `_spine/` (the shared to-do queues and coordination records that *can* change). The Growth Hacker reads its plan from the Spine and writes results to both.

There is a dedicated startup skill named **"pranaam"** whose whole job is to connect to Google Drive and verify it actually works before any real agent runs. Run it first. Think of it like a pre-flight handshake: it confirms the @iksula.com account is connected and that the agent can find the `_brain/` and `_spine/` folders.

One important detail the skill calls out: the agent finds the Brain folder by reading a small **seed file** called `brain_io-howto` and following its `parentId` (the folder's ID), because a plain folder-name search comes back empty on this Drive connection. You don't do this by hand — "pranaam" and the agent's built-in `brain_io` helper handle it. Just know that if pranaam reports it cannot find the Brain, that seed file is the thing to check.

Step 4 — Connect Zoho the safe way (never a shared password)

Zoho CRM holds real people's data, so it is connected with a proper, revocable connection — **never** by pasting a shared username and password into chat. There are two approved ways:

1. **Native Zoho MCP server** — a ready-made secure connector ("MCP" just means a standard, safe way for the AI to talk to an outside service). This is the preferred route.
2. **OAuth Self Client** — "OAuth" is the modern login-without-sharing-your-password method; a "Self Client" is Zoho's way to issue the agent its own keys. This is the fallback if the native connector isn't available.

Hard rule, no exceptions: **never** type a Zoho password (or any password) into the chat. Connect via MCP or OAuth only. Also make sure you connect to the **correct Zoho data centre** (the geographic region your Zoho account lives in), or the connection will simply fail to find your records.

Step 5 — Know exactly where secrets live (and where they must NEVER go)

The toolkit reads its switches and settings from **environment variables** — these are named values stored on the machine, *outside* of any file you share. The Growth Hacker's configuration code (`tools/growthhacker/config.py`) reads exactly one such variable:

- `GROWTH_PUBLISH` — the master kill-switch for publishing. Its default is `off`. Only the exact value `on` lets the agent publish; **anything else means HOLD everything**. (More on this in Step 8.)

The same config file also points at a few local working folders the toolkit uses on your machine. You do not normally touch these, but for completeness they are:

- `SPINE_QUEUE` → `tools/growthhacker/_state/content_sourced_lead_queue.jsonl` — the local "outbox" where interest events wait for Lead Gen to pick them up.
- `BRAIN_RAW_DIR` → `tools/growthhacker/_state/_brain_raw` — raw numbers, dumped first.
- `BRAIN_FEED_DIR` → `tools/growthhacker/_state/_brain_feeds` — the cleaned-up aggregate feeds.
- `AUDIT_PATH` → `tools/growthhacker/_state/audit_log.jsonl` — an append-only log helper (provided, but not yet wired into the gate/emitter in this version).

These `_state/` folders are **local mirrors** for now. In production the real Brain and Spine live on Google Drive and are reached through `brain_io`; the live distribution connectors (LinkedIn / Telegram / X) are **not built yet** — only a manual `telegram_publish.sh` helper exists, and it is not yet wired through the safety gate.

The golden rule for every secret (API keys, tokens, OAuth keys):

- Secrets live **only** in environment variables or in local files that are "gitignored" (a gitignored file is one Git is told to never upload or share).
- A secret is **never** pasted into chat, never committed to the code repository, and never written into the Brain. When you need to refer to a key, refer to it **by name** (e.g. "the Woodpecker API key"), never by its value.

Step 6 — Run the safety tests (prove the rails hold)

Before trusting the agent, confirm its safety code actually works. From inside the `tools/` folder, run:

```
python -m unittest growthhacker.tests.test_safety
```

This runs the toolkit's **28 safety tests**. They have been adversarially checked (deliberately attacked to try to break them) and found sound. They confirm things like: a paid post with no budget approval is blocked, a score secretly hidden inside a lead record is rejected, and no personal data can sneak into the Brain.

Step 7 — Do a dry run with preflight (no posting, no sending)

`preflight` is the "test the engine without leaving the driveway" command. It checks a sample publish action and a sample interest event and prints **ALLOW** or **HOLD** — without touching the real Brain, Spine, or any public platform (the sample event is written to a throwaway temporary queue and deleted afterward).

Run it like this (from the `tools/` folder):

```
python -m growthhacker.preflight --action action.json --event event.json
```

- `action.json` describes a pretend publish (its channel, the asset reference, the calendar status, whether it's instrumented, and so on).
- `event.json` describes a pretend "someone showed interest" event.

What you'll see in the output:

- `"publish_switch": "off"` or `"on"` — the current state of `GROWTH_PUBLISH`.
- A `publish_gate` verdict of **ALLOW** or **HOLD**, and if it's HOLD, the exact reasons (e.g. `publish_switch_off (set GROWTH_PUBLISH=on)`, `not_instrumented (UTM + tracked CTA required before publish)`, `paid_without_budget_approval`).
- A `seam_emit` result showing the interest event was accepted (and stamped with a lawful-basis tag) or rejected with a reason.

If the switch is off, expect to see the publish verdict come back **HOLD** with `publish_switch_off`. That is correct and healthy — it proves the kill-switch is doing its job.

Step 8 — Understand what is OFF by default (and leave it off until told)

This is the most important part. Out of the box, the Growth Hacker is deliberately **inert** — it can plan, reshape, draft, and dry-run, but it cannot push anything live. The switches and gates that keep it that way:

- `GROWTH_PUBLISH` **is** `off` **by default.** While it is off, **every** publish action is held. Nothing posts. Turning it to `on` is a deliberate operator decision, not a default.
- **Paid posts** are held until the Media Planner's **budget approval** is recorded (`budget_approved`). No approval → HOLD.
- **A reply posted as a named leader** (e.g. a comment under DJ's byline) is **always** held until a human supplies an `approval_token` to own that voice. The agent drafts the words; a human approves before they go out. This gate can never be switched on by the agent itself.
- **Any broadcast/marketing email the Growth Hacker might send** is held until a human approves the **first send**. The agent never sends at scale on its own.
- **Organic posts** (ordinary LinkedIn/X/blog/Telegram posts) have no separate human gate — but they still only run if the calendar row is marked `Scheduled` and the post is **instrumented** (carries its tracking tags), with the kill-switch on.

Today's correct setting, given the wider reality: leave `GROWTH_PUBLISH` **off**. The live posting connectors aren't built, Woodpecker is a live production account, and publishing stays switched off until DJ and the operators decide it's time. Setup is "done" when pranaam confirms the Drive connection, Zoho is connected via MCP/OAuth, the 28 tests pass, and a `preflight` dry run shows the gates behaving (HOLD while the switch is off). At that point the agent is ready — and safely idle — until someone deliberately flips it on.

6 How to run it — step by step, with a full worked example

This section is the operator's hands-on guide: what to type, what the agent does at each stage, how to test it safely first, and one full run from start to finish.

The trigger phrases you type

You "run" the Growth Hacker by typing a plain-English trigger into Claude Code (the chat window where the agent lives). The skill recognises any of these (all from the skill's own `description`):

- "run the growth hacker"
- "publish the calendar"
- "distribute this cycle's content"
- "schedule the posts"
- "run the distribution"
- "set up the A/B test" (A/B test = showing two versions of a post to see which performs better)
- "post this to LinkedIn"
- "capture this cycle's engagement"
- "draft the community replies"

You can also just say what you want in your own words. The phrases above are the reliable ones.

What the agent does, phase by phase (in plain steps)

When you trigger it, the agent walks an 8-phase checklist (the "Workflow" in its SKILL.md). In plain English:

Phase	What happens	Plain-English meaning
0. Intake & grounding	Reads the publishing calendar, media plan + budget, and the asset bundle from the Spine (the shared to-do/queue folder `_spine/`). Reads `channel-intel`, `icp-audience` (incl. the geography map), `competitor-radar`, `voices` from the Brain (the append-only knowledge folder `_brain/`).	It gathers the approved plan and the house rules. If a calendar row points at a missing post (`asset_ref`), or a paid row has no budget sign-off, it **STOPS and flags it** to the team channel `#agentic-org-requests` instead of guessing.
1. Schedule	Sets the exact post time per channel/timezone; moves the row's status from `Planned` to `Scheduled`.	Picks the clock slot. It does not change who the post targets or what it says.
2. Reshape (native, never rewrite)	Adapts the post to each platform's format — thread vs carousel, length, aspect ratio, hook placement.	Makes a LinkedIn post look like a LinkedIn post and an X post look like an X post — **without changing the claim, proof, or argument** (that is the Content Creator's). If it can't reshape without changing the argument, it flags it.
3. Instrument before publish	Attaches UTM tags (tracking labels added to a link so you can see where a click came from) and tracked call-to-action links.	Nothing ships untracked. This is how engagement is later tied back to the exact post.
4. Publish + distribute	Posts per the calendar; moves status to `Published`; writes an execution-log to the Spine as it goes. **Paid only:** must inherit the Media Planner's budget approval first.	Actually ships the posts (when switched on — see the kill-switch below). Organic posts have no extra human gate; paid spend does.
5. Run experiments	Runs A/B tests strictly within the plan's bounds — hooks, post times, audience cuts, CTAs. Pre-registers the hypothesis, the one primary metric, and the stop-rule; **no peeking**; declares a winner only at the stop-rule.	Tests two versions of an approved post, never two strategies.
6. Community engagement (voice-gated)	Drafts replies/comments. Anything posted as a named leader (a byline) is drafted against that person's `voices` persona and **needs a human to approve the voice** before it posts.	It writes replies in the leader's voice but a human owns/approves that voice. It does NOT send 1-to-1 private outreach (that is Lead Gen).

Phase	What happens	Plain-English meaning
7. Detect + emit interest (THE SEAM)	For each interest event (a comment, reply, DM on a public post, form fill, content download, tracked-CTA click, connection-accept, mention) it writes ONE `content-sourced-lead` record to the Spine queue Lead Gen polls. Each carries a unique `correlation_id` (a ULID — a sortable unique ID), the `source_asset_ref`, `source_channel`, the verbatim signal, and a `region` + `lawful_basis_tag` stamp.	When someone shows interest, the agent hands a raw note to Lead Gen. It never scores or grades it; it just records "this person did this, here, in this region."
8. Capture + return raw	Aggregates engagement by asset × format × channel × time. Dumps raw to `_brain/_raw/` first, then writes the two Brain feeds: `performance-analytics-growth-YYMMDD` (the numbers, aggregate, NO personal data) and `channel-intel-growth-YYMMDD` (what each channel rewarded + experiment winners). Writes the final execution-log + experiment-results to the Spine.	Reports the scoreboard back as numbers only. It never stores per-person data in the Brain.

The two rules to remember: **execute the plan, don't re-plan**, and **surface interest, don't qualify it**.

Before anything goes live: the safe dry-run (preflight)

The toolkit (`tools/growthhacker/`, the Python code that enforces the safety rules) ships a dry-run tool called **preflight**. It tests a publish action and a lead-handoff event **without posting anything and without touching the real Spine** — the seam event is written to a throwaway temp file that is deleted immediately after.

You run it from inside the `tools/` folder so Python can find the `growthhacker` package. On this machine that folder is:

`iksula-marketplace/plugins/iksula-agents/tools/`

Step 1 — write two tiny input files. A publish action and (optionally) a lead event, as JSON.

`action.json` (an organic LinkedIn post):

```
{
  "type": "organic_post",
  "channel": "LinkedIn-post",
  "asset_ref": "asset-2026Q2-carousel-07",
  "calendar_status": "Scheduled",
  "utm": "utm_source=linkedin&utm_campaign=2026q2",
  "tracked_cta": "https://iksula.com/x?cta=demo"
}
```

`event.json` (someone left a comment on that post, based in the EU):

```
{
  "source_asset_ref": "asset-2026Q2-carousel-07",
  "source_channel": "LinkedIn-post",
  "intent_signal": { "signal_type": "comment", "verbatim_text": "Do you support DPDP compliance?", "ob
served_at": "2026-06-22T09:14:00Z" },
  "contact_identity": { "platform_handle": "in/some-user", "display_name": "A. Buyer" },
  "region": "EU",
  "captured_at": "2026-06-22T09:14:05Z"
}
```

Step 2 — run preflight. The exact command lines (from `preflight.py`):

Test just the publish action:

```
python -m growthhacker.preflight --action action.json
```

Test the publish action AND the lead handoff together:

```
python -m growthhacker.preflight --action action.json --event event.json
```

Step 3 — read the PASS/HALT result. Preflight prints a JSON block and then a banner. The decisive line is `=== PUBLISH: ALLOW ===` or `=== PUBLISH: HOLD ===`. If it is HOLD, it lists exactly why under "Held for:". It also prints `"publish_switch": "on"` or `"off"` so you instantly see whether the master switch is on.

Two important details:

- The publish gate verdict is either **ALLOW** (every gate held) or **HOLD** (one or more gates failed). The tool's exit code is `0` for ALLOW and `2` for HOLD — handy if you script it.
- The **kill-switch** is the environment variable `GROWTH_PUBLISH`. It is **off** unless explicitly set to `on`. While it is off, *every* publish action returns HOLD with the reason `publish_switch_off (set GROWTH_PUBLISH=on)`. This is deliberate: the agent posts nothing until a human turns the master switch on for the session. To turn it on for one run (PowerShell): `$env:GROWTH_PUBLISH = "on"`.

The publish gate (in `publish_gate.py`) will HOLD a post for any of these, by design:

- `publish_switch_off` — the master switch is off.
- `not_instrumented` — missing UTM or tracked CTA.
- `calendar_row_not_scheduled` — the row isn't an approved `Scheduled` row.
- `paid_without_budget_approval` — a paid post with no Media Planner budget sign-off.
- `byline_voice_not_approved` — a reply as a named leader with no human approval token.
- `broadcast_first_send_not_human_approved` — a broadcast email with no human first-send approval.
- `no_asset_ref` / `unknown_action_type` / `channel_not_allowed` — malformed or off-scope action.

The seam emitter (in `seam_emitter.py`) will likewise **reject** a lead handoff if it carries any forbidden field — anything that looks like a score, grade, MQL/SQL, tier, rating, ack_status, account_status, or enrichment — even if you try to hide it nested inside another field. Growth Hacker is structurally unable to qualify a lead.

One full worked example, start to finish

The setup. It is a normal cycle in June 2026. The Spine holds an approved publishing calendar with one organic LinkedIn carousel (`asset_ref = asset-2026Q2-carousel-07`) due this morning, plus a draft reply for the company's named thought-leader. The operator (say, Vishal) types:

run the growth hacker

Phase 0 — intake. The agent reads the calendar, the media plan, and the asset bundle from `_spine/`, and the Brain modules (`channel-intel`, `icp-audience` with the geography map, `competitor-radar`, `voices`). Every calendar row resolves to a real asset and has a channel and slot. Nothing is missing, so it proceeds (if something were missing it would stop and post to `#agentic-org-requests`).

Phases 1–3 — schedule, reshape, instrument. It sets the post for the LinkedIn business-hours window, reshapes the asset into a native LinkedIn document carousel with the hook in the first two lines (no wording of the argument changed), and attaches the UTM tag and a tracked demo-CTA link. The row moves `Planned → Scheduled`.

The gate check. Before posting, the operator runs the dry-run to be sure:

```
python -m growthhacker.preflight --action action.json
```

With `GROWTH_PUBLISH=on`, the carousel action is fully instrumented and on a `Scheduled` row, so the result is:

```
=== PUBLISH: ALLOW ===
```

Phase 4 — publish. The organic post ships (organic has no extra human gate). The agent writes an execution-log line to the Spine: asset, channel `LinkedIn-post`, timestamp, status `Published`, no deviations. Status moves `Scheduled → Published`.

Phase 6 — the byline reply is HELD. A buyer comments on the post and the leader could reply. The agent drafts the reply in the leader's `voices` persona — but a reply as a named leader (`community_reply_as_byline`) **always** needs a human approval token. With no token, the gate returns:

```
=== PUBLISH: HOLD ===
Held for: byline_voice_not_approved (a human owns the voice - draft, never post unapproved)
```

So the draft sits and waits for the leader (or the operator on their behalf) to approve the voice. Nothing posts as that person until then. This is correct behaviour, not an error.

Phase 7 — a lead is captured (the happy path). That same comment is an interest event. The agent emits ONE `content-sourced-lead` to the Spine queue. Because the commenter's region is EU, the emitter fails safe and stamps `lawful_basis_tag = "non-US: company-level ONLY (GDPR/DPDP)"`. The record carries a fresh ULID `correlation_id`, the verbatim comment text, the source post and channel — and crucially **no score and** `ack_status: null` (Lead Gen fills that in when it consumes the record). The dry-run of this event prints:

```
"seam_emit": { "status": "emitted", "correlation_id": "01J...", "lawful_basis_tag": "non-US: company-level ONLY (GDPR/DPDP)" }
```

If that same event is emitted twice (e.g. a retry), the second emit returns `"status": "duplicate"` — it is idempotent, so Lead Gen never sees the lead twice.

Phase 8 — return the numbers. At the end of the cycle the agent aggregates engagement (impressions, reactions, comments, clicks, saves) by asset × format × channel × time, stamps `audience_type` (new vs retargeting), dumps the raw to `_brain/_raw/` first, then writes `performance-analytics-growth-260622` and `channel-intel-growth-260622` to the Brain — **as numbers only, no names, no emails**. It also writes the final execution-log and any experiment-results to the Spine. The count of content-sourced leads ("3 from LinkedIn-post this cycle") goes to the Brain as a number; the actual lead rows stay in the Spine queue for Lead Gen.

Outcome of this run: one organic post published and instrumented; one byline reply correctly held for human voice approval; one EU lead handed to Lead Gen verbatim, company-level only; the scoreboard returned to the Brain with no personal data in it.

How this compares to a Lead Gen run today

If instead you ran **Lead Gen** today, the realistic ending is different. Lead Gen is the agent that would actually email people — and "lawful basis" (recorded permission to email a given person) is set **nowhere in Zoho yet**, and no warmed sending mailbox is provisioned. So the correct number of emails to send today is **ZERO**, and a Lead Gen run ends in **HOLD** until DJ signs the LIA (Legitimate Interest Assessment) and a sending mailbox is set up. The Growth Hacker, by contrast, can complete a real run today — it publishes already-approved 1-to-many content and only ever *hands off* interest; it never emails an individual. Its only "off by default" piece is the `GROWTH_PUBLISH` switch, which a human flips on per session.

7 Guardrails & safety, current limits, and troubleshooting

The Growth Hacker agent runs on a "safe-by-construction" design. That is a fancy way of saying: the dangerous actions are blocked by the computer code itself (the "toolkit" under `tools/growthhacker/`), not just by polite instructions in the agent's playbook. So even if the AI got confused and *tried* to do something risky, the code would refuse. This section explains, in plain terms, what it will and won't do, what stops it cold today, what is deliberately not finished yet, and what to do when something looks stuck.

What it WILL do vs. what it WON'T do

The Growth Hacker WILL...	The Growth Hacker WON'T...
Publish content that is **already approved** (the publishing calendar + media plan handed over by Content Creator and Media Planner)	Re-plan, re-write, or re-argue the content. It reshapes an asset to fit a platform (length, aspect ratio, thread vs. carousel) but **never changes the claims or the argument**
Reshape one approved post into each platform's native format (LinkedIn, X, YouTube, blog, Telegram)	Create new content. That is the Content Creator's job
Instrument every post before it ships — attach UTM tags (the little tracking codes added to a link so you can tell which post a click came from) and tracked call-to-action links	Ship anything uninstrumented. No tracking = the code blocks the publish
Run A/B tests (try two versions and see which wins) **only within the plan's limits** — on hooks, post times, audience cuts, and call-to-action wording	Experiment on the thesis or the channel mix. Those belong to the plan/Media Planner

The Growth Hacker WILL...	The Growth Hacker WON'T...
Draft community replies in a leader's voice for a human to approve	Post a reply *as a named person* without a human approving the voice first
Capture who engaged (comment, reply, DM, form fill, download, link click, connection accept, mention) and hand each event to Lead Gen, raw and verbatim	**Qualify** anyone — no scoring, no grading, no "MQL/SQL" labels, no de-anonymizing a visitor, no looking up the CRM. That is all Lead Gen's job
Stamp each handed-over event with a **region** and a **lawful-basis tag** (a note about what permission rules apply, e.g. US vs. Europe/India)	Send 1-to-1 outreach, personalized DMs, or nurture emails. Same platforms, opposite motion — that is Lead Gen
Write **aggregate numbers only** (counts, totals) back to the Brain (the shared, append-only knowledge folder)	Write any personal data (names, emails, handles, profile URLs) into the Brain — the code rejects PII-looking rows
Flag a problem (missing asset, no budget approval, channel glitch) to the team channel and stop	Silently improvise or "fix" a broken plan on its own

The kill-switch — and it is OFF by default

There is one master on/off switch, an environment variable (a setting stored outside the code) named `GROWTH_PUBLISH`.

- Its default value is `off`. While it is off, **every** publish action is held back — nothing posts, full stop.
- It only permits publishing when explicitly set to exactly `on` (`GROWTH_PUBLISH=on`).
- Today it is off. So the honest answer to "is the Growth Hacker posting anything live?" is **no**.

Think of it like the master breaker in a fuse box: until someone deliberately flips it on, the whole publishing circuit is dead.

The gates the agent literally cannot bypass

Beyond the master switch, four specific checks (called "gates") sit in `publish_gate.py`. Every publish request is run through `evaluate()`, which returns either **ALLOW** or **HOLD** with a list of reasons. A request only goes through if **every** check passes:

1. **Master switch must be on.** If `GROWTH_PUBLISH` is not `on`, it holds with `publish_switch_off (set GROWTH_PUBLISH=on)`.
2. **Paid posts need budget approval.** A paid post is held with `paid_without_budget_approval (Media Planner gate)` unless the Media Planner's spend approval is confirmed. The Growth Hacker never approves spend itself.
3. **A reply posted as a named leader needs a human "voice" approval.** Held with `byline_voice_not_approved` unless a human approval token is present. Important: this gate is **always on for that action type** — the agent cannot turn it off by passing a flag.
4. **A broadcast/BOFU email needs a human "first-send" approval.** Held with `broadcast_first_send_not_human_approved` unless a human has signed off on the first send. The agent never autonomously emails at scale.

Two more housekeeping checks also apply: the post must be **instrumented** (UTM + tracked CTA) or it holds with `not_instrumented`, and an organic/paid post must be executing an approved calendar row whose status is exactly `Scheduled`, or it holds with `calendar_row_not_scheduled`. Organic posts have **no separate human gate** (the calendar and plan were already human-approved upstream) — but they still need the master switch on, instrumentation, and a Scheduled row.

Why the agent HALTS today (the real, current reasons)

Even with the perfect plan in hand, the agent is correctly stopped right now for these reasons:

- **The master switch is off.** `GROWTH_PUBLISH` defaults to `off`, so every publish is held. This is the single biggest reason nothing goes live.
- **No live publishing connectors are wired yet.** The actual "plumbing" to LinkedIn, X, and Telegram is not built (see below). A Telegram helper script exists but is not yet routed through the gate.
- **Anything involving email is human-gated by design.** If the Growth Hacker ever sent a broadcast email, the first send waits for a person. And separately, across the whole iKshana system, **lawful basis (recorded permission to email**

someone) is set nowhere yet and there is **no warmed sending mailbox** — so the correct number of emails to send today is **zero**. The Growth Hacker's main job is organic posting, not email, but this is why nothing email-shaped moves.

- **Named-leader replies wait for a human voice approval.** No approval token = held.
- **A broken or incomplete plan stops the run.** If a calendar row has no real `asset_ref`, an unmet dependency, or a paid row with no budget approval, the agent stops and flags it rather than improvising.

What is intentionally NOT built yet (and why)

These are honest gaps, listed in the toolkit's own README — they are deferred on purpose, not bugs:

- **Live distribution connectors** (the real send-to-LinkedIn / X / Telegram code) behind the gate. A manual Telegram helper (`telegram_publish.sh`) exists, but wiring it through the publish gate is the next piece of work. Until then, "publish" produces drafts/decisions, not live posts.
- **Real Brain/Spine transport.** Today the toolkit writes to local practice files under a `_state/` folder on disk (a safe sandbox). The production version will read and write the shared Google Drive `_spine/` and `_brain/` folders via the `brain_io` connector. So results currently land in a local mirror, not the live shared system.
- **Voices register integration.** Right now the human approval token *is* the voice gate. Automatically matching a drafted reply to the correct leader's persona in the Brain's `voices` file is a follow-up step.

This mirrors how the system is meant to work: build the safety rails first, switch on the live actions later, deliberately.

How to do a safe dry-run (no posting, no sending)

There is a built-in "test fire" you can run that checks a publish action and a handoff event **without touching anything real**:

```
python -m growthhacker.preflight --action action.json --event event.json
```

This prints whether the master switch is on or off, and an `ALLOW`/`HOLD` verdict with the exact reasons. The handoff event is written to a temporary throwaway file, so the real Spine queue is never touched. There is also a safety test suite (`python -m unittest growthhacker.tests.test_safety`, 28 tests) that has been adversarially checked and reported sound.

Troubleshooting / FAQ

1. "I ran it and nothing got posted."

That is expected today. The master switch `GROWTH_PUBLISH` is off by default, and the live publishing connectors are not wired yet. The agent will have produced drafts, decisions, and an execution log — but it will not push anything live. To genuinely turn publishing on you would set `GROWTH_PUBLISH=on` **and** have the connectors built.

2. The preflight says HOLD: `not_instrumented`.

The post is missing its tracking. Every post needs both a UTM tag and a tracked call-to-action link before it can ship. Add them and re-run.

3. The preflight says HOLD: `calendar_row_not_scheduled`.

The agent only executes approved plan rows whose status is exactly `Scheduled`. The row you pointed it at is still `Planned` (or already `Published`). Make sure the calendar row has been advanced to `Scheduled` first.

4. The preflight says HOLD: `paid_without_budget_approval`.

You asked it to run a paid post, but the Media Planner's budget approval is not recorded. The Growth Hacker never approves its own spend — get the budget approval confirmed on that row, then retry.

5. The preflight says HOLD: `byline_voice_not_approved` (or `broadcast_first_send_not_human_approved`).

You asked it to reply as a named leader, or to send a broadcast email, without a human sign-off. These are deliberate human gates. A person must approve the voice / the first send. Provide the approval, then retry. This is working as designed, not a fault.

6. "The handoff to Lead Gen was rejected."

The seam emitter (`seam_emitter.emit`) refuses any handoff record that carries a score, grade, MQL/SQL label, ack status, account status, or enriched/de-duped data — even if it is hidden inside a nested field. It also requires a valid signal type (comment, reply, DM, form fill, download, click, connection accept, mention), a `source_asset_ref`, an allowed channel, and a `captured_at` timestamp. Strip out any scoring/qualification fields and supply the required raw fields. Qualification is

Lead Gen's job, never the Growth Hacker's.

7. "It wrote a duplicate lead to Lead Gen."

It won't. Each handoff has a unique ULID `correlation_id`. If the same event is sent twice, the second one is a no-op (it returns `duplicate` and writes nothing). Re-running is safe.

8. "The Brain write was rejected as PII."

The Brain only accepts aggregate numbers on a fixed schema (`period | source | asset_or_campaign | channel | metric | value | audience_type | notes`). A row that includes a name, email, handle, profile URL, phone, IP, or click ID — or any value that looks like an email or @handle — is refused. Roll your numbers up into counts/totals (no per-person rows) and write those instead. Per-person data belongs in the Lead Gen seam (the Spine), never in the Brain.

9. "How do I confirm the switch state right now?"

Run the preflight (above) — the first line of its output is `"publish_switch": "off"` or `"on"`. If you did not deliberately set it, it will read `off`.

8 Glossary — every term in plain English

This is a plain-English dictionary for the Growth Hacker agent. Each term gets one simple sentence. They are grouped by theme (the big picture first, then the safety rules, then the technical bits), and alphabetised inside each group.

The big picture — who and what

- **iKshana** — Iksula's "AI-native" marketing-and-sales organisation: a chain of AI agents that hand work to each other, end to end.
- **Agent** — one AI worker with a single job (the Growth Hacker is one agent; Lead Gen is another).
- **Skill** — the markdown (plain-text-with-formatting) instruction sheet that tells the AI agent how to behave; for this agent it is `SKILL.md`.
- **Toolkit** — the Python computer code (under `tools/growthhacker/`) that *enforces* the safety rules, so the agent literally cannot do the forbidden thing even if it wanted to.
- **Growth Hacker (GH)** — the agent that takes already-approved content and *distributes* it (posts it to LinkedIn, X, YouTube, blog, Telegram, newsletter), runs the planned experiments, captures who engaged, and hands interested people to Lead Gen.
- **Lead Gen** — the next agent in the chain; it takes the interested people GH found and does everything afterwards (identifying them, checking permission, scoring, follow-up).
- **Sales / KAM** — the humans at the very end who close deals; KAM means Key Account Manager (the person who looks after an existing client).
- **Media Planner** — the upstream agent that decides which channels to use, how much money to spend, and the schedule; GH obeys that plan, never rewrites it.
- **Content Creator** — the upstream agent that writes the actual posts and articles; GH reshapes them for each platform but never changes the message.
- **DJ / Vishal / Yatin** — the people: DJ is the founder/visionary, Vishal is the operator (and Woodpecker/Zoho admin), Yatin is the builder.
- **Pilot** — the current early test phase; "the conductor isn't live in the pilot" means the automatic hand-off machinery isn't running yet, so a human runs and approves steps by hand.

The architecture — where everything lives

- **Brain** — a shared, *append-only* (you can add but never change or delete) Google Drive folder called `_brain/` that holds general knowledge and *aggregate* (summed-up) numbers, and never any personal data.
- **Spine** — a shared Google Drive folder `_spine/` that holds the live to-do queues, the calendar, and coordination records; unlike the Brain, things here *can* change.
- **Hands** — the agents themselves, the workers that actually *do* things (GH is "the last fast-loop Hand").

- **Zoho CRM** — the separate "system-of-record" (the official master list) that stores the per-person lead details; only Lead Gen writes to it, not GH.
- **Seam** — the single, one-way hand-off point from GH to Lead Gen; GH drops one record per interested person into it and Lead Gen picks it up.
- **content-sourced-lead** — the name of the record GH writes onto the seam: one per detected interest event, raw and unjudged.
- **content-sourced-lead queue** — the actual list/file in `_spine/` (named `content_sourced_lead_queue.jsonl` in the local mirror) where those records pile up for Lead Gen to read.
- **Execution log** — the running record GH writes to the Spine of what it published, when, on which channel, and any deviation from the plan.
- **Experiment-results record** — the Spine record listing each A/B test's variants, the winner, and the lift.
- **brain_io** — the helper that GH uses to read from and write to the Brain folder (it resolves folder locations at runtime; GH never hard-codes them).
- **brain_io-howto** — the live instruction file inside `_brain/` that tells the agent how to use `brain_io` correctly (e.g. read feeds with `download_file_content`, not `read_file_content`, which would corrupt the data).
- **Brain feed** — a dated file GH appends to in the Brain; GH owns exactly two: `performance-analytics-growth-YYMMDD` and `channel-intel-growth-YYMMDD`.
- **performance-analytics-growth-YYMMDD** — GH's Brain feed of raw engagement numbers (impressions, clicks, etc.), aggregate-only, no personal data.
- **channel-intel-growth-YYMMDD** — GH's Brain feed summarising what each channel rewarded this cycle plus the experiment winners.
- **Namespaced** — each feed is "owned" by one writer under its own name-tag (the `source` field), so two agents never overwrite each other's numbers.
- **Append-only** — you may only add new lines; you can never edit or erase what's already there (the Brain's core rule).
- **Raw-first** — the discipline of dumping the unprocessed numbers into `_brain/_raw/` *before* publishing any summarised figure, so every figure is traceable back to its source.

The inputs GH reads

- **Publishing calendar** — the schedule (in the Spine) of which asset goes out on which channel at which slot; GH executes it, never owns it.
- **Media plan + budget** — the Spine record of channels, spend, schedule, and KPIs (Key Performance Indicators — the target numbers) that GH must follow exactly.
- **Asset bundle / content package** — the produced content (in the Spine), each piece addressable by its `asset_ref`.
- **asset_ref** — the unique reference/ID that points to one specific piece of content.
- **channel-intel** — Brain knowledge about each platform's posting norms and what it rewards this quarter (used for timing and native formatting).
- **icp-audience** — Brain knowledge about who to target, plus the *geography map* (US core vs EU/India/other) that decides the region tag on every seam record.
- **ICP** — Ideal Customer Profile: the description of the kind of company/person Iksula wants as a customer.
- **competitor-radar** — Brain knowledge about rivals' timing, used to find posting *whitespace* (gaps rivals leave open).
- **voices** — the Brain register of the byline personas (the writing style/identity of each named leader) that governs how community replies sound.

The jobs GH does

- **Distribution** — the act of getting approved content out onto the channels; this is GH's core job.
- **1-to-many** — broadcasting to a crowd (a public post everyone can see); GH only does this, never 1-to-1.
- **1-to-1** — a personal, private message to one named person (a cold DM or personalised email); this is Lead Gen's job, never GH's.

- **Organic** — free, unpaid posting (a normal LinkedIn post); it needs no separate human gate but must still be instrumented and scheduled.
- **Paid distribution** — putting money behind content (ads/boosted posts); GH may only run it after the Media Planner's budget approval is confirmed.
- **Retargeting** — paying to re-show content to people who already engaged once; GH runs it only within the plan's budget caps.
- **Reshape, never rewrite** — GH may change the *form* of an asset (length, aspect ratio, thread vs carousel, hook placement) but never the claim, proof, thesis, or message.
- **Native** — in the natural form a platform expects (a thread on X, a carousel on LinkedIn) so it doesn't look copy-pasted.
- **Hook** — the first line or headline that grabs attention; GH may A/B test it.
- **CTA** — Call To Action: the "click here / download / book a demo" prompt; GH attaches a *tracked* one so clicks can be counted.
- **Thread / carousel** — a thread is a chain of linked posts (X); a carousel is a swipeable multi-image/slide post (LinkedIn).
- **Byline leader / named byline** — a real, named person whose name is on the post (a "thought leader"); posting or replying as them needs human voice approval.
- **Voice gate** — the hard rule that anything posted as a named leader must first be approved by a human who owns that voice; GH drafts, never invents it.
- **Community engagement** — replying, commenting, and monitoring on public posts (1-to-many); this is GH's, but a private DM to a named prospect is not.
- **Social listening** — always-on monitoring of comments, mentions, and DMs to spot interest, without judging or scoring it.
- **Employee advocacy** — offering pre-approved reshare versions of a post to opted-in staff so it spreads through their networks (opt-in only, no auto-posting).
- **Experiment / A/B test** — running two versions (A vs B) of the same approved asset to see which performs better, varying only the allowed things (hooks, post times, audience cuts, CTAs).
- **Hypothesis** — the single testable statement written down *before* an experiment starts.
- **Primary metric** — the one success measure chosen up front for an experiment (e.g. saves, click-through), so no one cherry-picks results afterwards.
- **Stop-rule** — the pre-set sample size and stopping point; a winner is declared only when it's reached.
- **No peeking** — the rule against checking results early and stopping when they look good, which would produce false winners.
- **Lift** — how much better the winning variant did than the other, expressed as the improvement.
- **audience_type** — the always-stamped label "new vs retargeting", kept constant within a comparison so results are read honestly.
- **Instrument / instrumentation** — attaching tracking to a post *before* it ships so engagement can be measured and tied to its asset.
- **UTM** — the little tracking tags added to a link so you can tell which post/campaign a click came from (Urchin Tracking Module, the web-standard label).
- **CTOR** — Click-To-Open Rate: for email, the share of *openers* who clicked; GH prefers it over plain CTR for owned email.
- **Whitespace** — open time slots/gaps competitors leave unused, where GH can post to stand out.

The interest hand-off (the seam in detail)

- **Interest event** — a single observed action that signals interest (one comment, one form fill, one tracked click); GH emits one seam record per event.
- **signal_type** — the kind of interest event, from a fixed list: `comment`, `reply`, `dm`, `form_fill`, `content_download`, `cta_click`, `connection_accept`, `mention`.
- **intent_signal** — the seam field holding the raw, verbatim action and its text — with no score attached.
- **Verbatim** — recorded exactly as observed, word-for-word, with nothing added or judged.
- **Unqualified / unscored** — handed over without any judgement of how good the lead is; GH never grades.

- **contact_identity** — whatever GH actually saw of the person (a handle, a profile URL, an email only if they volunteered it), partial and un-cleaned.
- **correlation_id** — the unique ID stamped on each seam record (a ULID) that ties the event back through the campaign chain and prevents duplicates.
- **ULID** — Universally Unique Lexicographically Sortable Identifier: a unique, time-ordered code used here as the seam's idempotency key.
- **Idempotent / idempotency** — safe to do twice: if the same record is delivered again, the second delivery does nothing (a no-op).
- **No-op** — "no operation"; an action that deliberately does nothing (what a duplicate emit becomes).
- **At-least-once delivery** — the record might be sent more than once on retry; idempotency makes the repeats harmless.
- **region** — the contact's best-known location, mapped against the icp-audience geography map and stamped on the seam.
- **lawful_basis_tag** — the permission label GH derives from `region` (US → person-level allowed downstream; EU/India/other/unknown → company-level only).
- **captured_at** — the exact UTC timestamp (ISO-8601 format, the international date-time standard) of when the interest was detected.
- **consent_context** — any opt-in GH actually saw at capture (a ticked form box, a newsletter sign-up), or empty.
- **emitted_by** — the provenance stamp `growth-hacker`, recording which agent wrote the record.
- **ack_status** — the acknowledgement field (`received` / `de-duped` / `rejected`) that *Lead Gen* fills in on consume; GH always leaves it empty.
- **Provenance** — the recorded trail of where something came from and who created it.

The jobs that are NOT GH's (owned by Lead Gen)

- **MQL** — Marketing Qualified Lead: someone judged good enough for marketing to nurture; GH never assigns this.
- **SQL** — Sales Qualified Lead: someone judged ready for a salesperson; GH never assigns this either.
- **Qualify / scoring / grading** — judging how good or ready a lead is; entirely Lead Gen's, never on the seam.
- **De-anonymise (de-anon)** — turning an unknown website visitor into a named person/company; Lead Gen's job, and only where law allows.
- **Enrichment** — filling in missing details about a contact from outside data sources; Lead Gen's job.
- **Dedup (de-duplicate)** — merging repeat records of the same person into one master record; Lead Gen's job, in the CRM.
- **Suppression** — the "do-not-contact" scrub that removes people you must not email (opt-outs, current clients) before any send; Lead Gen runs it.
- **Merge field** — a placeholder in an email like "Hi {{first_name}}" that gets filled with each person's real detail at send time; part of Lead Gen's 1-to-1 outreach, not GH's.
- **Nurture / drip sequence** — a planned series of follow-up messages over time; Lead Gen's, not GH's.
- **ABM** — Account-Based Marketing: targeting specific named companies as a unit; Lead Gen's.
- **Account-status routing / new-vs-ECS** — deciding whether a contact is a new prospect or an Existing Customer/Client (ECS) and routing accordingly; Lead Gen only, because GH can't see CRM state.
- **Reference channel** — leads that come pre-qualified from delivery/relationship touchpoints; they bypass the seam entirely and never go through GH.

The law and permission

- **Lawful basis** — the recorded legal reason you're allowed to email or contact someone; today it is set *nowhere* in Zoho, so the correct number of emails to send is ZERO until DJ signs off.
- **LIA** — Legitimate Interest Assessment: the document DJ must sign to establish a lawful basis before outreach can begin.
- **GDPR** — the European Union's data-protection law; under it, EU contacts are company-level only here.
- **DPDP** — India's Digital Personal Data Protection law; under it, Indian contacts are company-level only here.
- **CAN-SPAM** — the US law governing commercial email (requiring honest headers, a real address, and a working unsubscribe).

- **List-Unsubscribe / one-click unsubscribe** — the standard way a recipient can opt out of an email in one click; required on any broadcast email.
- **PII** — Personally Identifiable Information: data that identifies a real person (name, email, handle, phone); banned from the Brain, allowed only on the Spine seam.
- **Fail-safe** — defaulting to the safest, most restrictive setting when unsure (e.g. an unknown region is treated as company-level only).
- **Deliverability** — how reliably your emails actually land in inboxes rather than spam; protected by warm-up and good sending hygiene.
- **Warm-up** — gradually building a new sending mailbox's reputation by ramping volume slowly, so its emails don't get marked as spam; a warmed mailbox must exist before any sending.

The safety machinery (the toolkit)

- **Gate** — a checkpoint that returns ALLOW or HOLD; a publish only happens if its gate holds.
- **publish_gate** — the toolkit module (`publish_gate.evaluate()`) that decides ALLOW/HOLD for every publish but never actually posts anything.
- **HOLD / ALLOW** — the gate's two verdicts; HOLD means "blocked, here's why", ALLOW means "all checks passed".
- **The three gates** — paid spend needs `budget_approved`; a named-byline reply needs a human `approval_token` (voice gate); a broadcast email needs a human first-send `approval_token`.
- **approval_token** — the proof that a human has approved a voice or a first send; without it the gate HOLDs.
- **budget_approved** — the flag (inherited from the Media Planner) that a paid post's spend was approved; without it, paid posts HOLD.
- **calendar_status / Scheduled** — a calendar row must be in "Scheduled" state for GH to publish it; statuses move Planned → Scheduled → Published.
- **Create-only** — GH (and Lead Gen) may build and draft things but never press the final send/run button; a human does that.
- **Kill-switch** — the master off-switch; here the environment variable `GROWTH_PUBLISH` must be set to `on`, or every publish is held.
- **Environment variable (env var)** — a setting read from the computer's environment (like `GROWTH_PUBLISH`) rather than written in the code; secrets and switches come from here only.
- **seam_emitter** — the toolkit module (`seam_emitter.emit()`) that writes a content-sourced-lead correctly: ULID, lawful-basis stamp, and it *rejects* any score/grade/ack/account field, even one hidden deep inside.
- **FORBIDDEN_SEAM_FRAGMENTS** — the built-in blocklist of word-fragments (`score`, `grade`, `mql`, `sql`, `qualif`, `enrich`, `tier`, etc.) that the emitter refuses to let cross the seam.
- **Recursive rejection** — the check that scans not just the top level but every nested layer of a record, so a banned field can't be smuggled inside another field.
- **brain_metrics** — the toolkit module (`write_growth_metrics()`) that writes the Brain feed, locked to a fixed schema and rejecting any row that looks like PII.
- **Schema / schema-locked** — the fixed set of columns a record must have; here `period | source | asset_or_campaign | channel | metric | value | audience_type | notes`, and nothing else is allowed.
- **preflight** — the dry-run tool (`python -m growthhacker.preflight`) that tests a publish action and a seam emit safely, touching a temporary queue, posting nothing.
- **Dry-run** — a practice run that checks what *would* happen without actually doing it.
- **Safety tests** — the 28 automated checks (`python -m unittest growthhacker.tests.test_safety`) that prove the rails hold; the toolkit was adversarially verified as sound.
- **Adversarially verified** — deliberately attacked/probed by someone trying to break the rules, and confirmed to hold up.
- **Audit log** — the toolkit's PII-scrubbed record (`audit.py`) of who published or emitted what and which gate decisions were made.
- `_state/` — the local folders where the toolkit writes practice mirrors of the Spine queue and Brain feeds today; production will swap these for the real Drive `_spine/` and `_brain/` via brain_io.

General technical terms

- **MCP** — Model Context Protocol: the standard "plug" that lets an AI agent connect to an outside service (like Zoho CRM); "at the MCP-wrap" means once the seam becomes a real connected queue rather than a by-convention file.
- **Connector** — a built integration that links the agent to a live external tool (e.g. a LinkedIn or Telegram connector); these are not built yet, so GH does not actually post live today.
- **Ledger** — a running, append-only record of actions taken (used elsewhere in the system to make sure nothing dangerous runs unchecked).
- **By convention** — done by an agreed manual practice rather than by automated machinery; in the pilot, the seam record is written this way and a human/Lead Gen polls for it.
- **Poll / polling** — repeatedly checking a queue for new items (how Lead Gen picks up the seam records).
- **JSON / JSONL** — JSON is a structured text format for data; JSONL is one JSON record per line, the format of the seam queue file.
- **CSV** — Comma-Separated Values, a simple spreadsheet-style text format; the Brain performance feeds are written as CSV.
- **Trigger** — the phrase you type to start the agent, e.g. "run the growth hacker", "publish the calendar", or "distribute this cycle's content".
- **Funnel** — the journey from first awareness down to a closed sale; GH owns no funnel and computes no funnel conversion (that's the Brain/Lead Gen).
- **TOFU / MOFU / BOFU** — Top / Middle / Bottom Of Funnel: how close a piece of content is to a purchase decision; GH never changes a piece's funnel intent.
- `${CLAUDE_PLUGIN_ROOT}` — the placeholder for the plugin's own folder path, used so file paths work wherever the plugin is installed, instead of hard-coding locations.

9 Fine print, known rough edges & extra answers

Small but important details that did not fit neatly above. Read this once before you operate the agent.

Why "Media Planner" appears twice in the line

The pipeline reads *Thought Leadership → Media Planner → Content Creator → Media Planner → Growth Hacker*. That second Media Planner is **not a typo**: the Media Planner first sets the rough plan, and then — *after* the Content Creator has actually produced the assets — does a second pass to finalise the real schedule and channel mix around the finished content. Only then does the Growth Hacker publish.

Exactly when the seam refuses an interest event

`seam_emitter.emit()` will **reject** an event (raise an error instead of writing it) if any of these are true:

- it carries a forbidden "qualification" field (even hidden inside `intent_signal` or `contact_identity`) — see the blocklist below;
- the `signal_type` is missing or not one of the allowed actions (comment, reply, dm, form_fill, content_download, cta_click, connection_accept, mention);
- `source_asset_ref` is missing; `source_channel` is not an allowed channel; `captured_at` is missing;
- a `correlation_id` is supplied but is **not a valid ULID**.

Dedupe gotcha: if you do **not** supply a `correlation_id`, the emitter mints a **fresh** one every time. So two separate captures of the *same* comment will **not** be recognised as duplicates unless the caller passes the *same* `correlation_id` both times. Dedupe protects against the *same ticket delivered twice*, not against *two tickets describing the same comment*.

The full "forbidden field" blocklist (and how matching works)

The agent must never put qualification/scoring/CRM-routing data on the seam (that is Lead Gen's job). The check is a **normalized substring scan** — it lower-cases the key and strips punctuation, then blocks the key if it contains any of:

`score, grade, mql, sql, qualif, ackstatus, accountstatus, isecs, enrich, disposition, propensity, tier, rating, ranking, leadstage, routeto`.

So `Lead_Score`, `MQL` , `lead-grade`, `routeTo`, `propensity` etc. are all caught.

Field-name gotcha: `asset_ref` vs `source_asset_ref`

A **publish action** (what you feed `publish_gate`) uses `asset_ref`. A **seam event** (what you feed `seam_emitter`) uses `source_asset_ref`. They are different keys — do not copy one JSON's field name into the other, or you will hit a validation error.

Which region strings count as "US"

Only these exact tokens (case-insensitive) map to **person-level** allowed: `US`, `USA`, `UNITED STATES`, `U.S.`, `U.S.A.`. **Anything else** — including `America`, `United States of America`, or an empty value — falls through to **company-level only** (the safe default). If you mean USA, use one of those exact tokens.

Settings vs fixed folders

Only `GROWTH_PUBLISH` is an environment variable you set (it must equal `on` to allow any publish; anything else = everything is held). The other paths the code mentions — `SPINE_QUEUE`, `BRAIN_RAW_DIR`, `BRAIN_FEED_DIR`, `AUDIT_PATH` — are **fixed folder locations the code computes for itself**, not settings you tune.

Reading preflight correctly

`python -m growthhacker.preflight` returns exit code `0` on ALLOW and `2` on HOLD. **Watch out:** if you pass only `--event` (a seam test) and **no** `--action` (a publish test), the publish verdict defaults to ALLOW (exit 0) — that does **not** mean a post was approved; it means you didn't ask it to check a post. Run it from the `tools/` folder (see below).

A complete, copy-paste publish action

```
{
  "type": "organic_post",
  "channel": "LinkedIn-post",
  "asset_ref": "PC2-post-01",
  "calendar_status": "Scheduled",
  "utm": "utm_source=li&utm_campaign=pc2",
  "tracked_cta": "https://iksula.co/x?c=1"
}
```

To unblock a **named-byline community reply**, the action must include a human approval token, e.g. `"type": "community_reply_as_byline"`, `"approval_token": "approved-by-DJ"` — without that token the reply is held.

Where to run things (Windows)

The toolkit lives at `iksula-marketplace/plugins/iksula-agents/tools/`. Open a terminal **in that folder** before running tests or preflight, so Python can find the `growthhacker` package:

- PowerShell: `Set-Location "iksula-marketplace/plugins/iksula-agents/tools"` then `$env:GROWTH_PUBLISH = "on"; python -m growthhacker.preflight --action action.json`
- Git Bash: `cd iksula-marketplace/plugins/iksula-agents/tools` then `GROWTH_PUBLISH=on python -m growthhacker.preflight --action action.json`

The 28 safety tests run the same way: `python -m unittest growthhacker.tests.test_safety` (from that `tools/` folder).

Quick glosses for a few terms used above

- **BOFU** — "bottom of funnel": a message aimed at someone almost ready to buy.
- **Namespace / namespaced** — one owner writes under its own labelled section so writers never clash (e.g. the Growth Hacker's Brain pages are tagged `-growth-`).
- **Provenance** — a stamp saying where a record came from (here, `emitted_by = growth-hacker`).

- **CTOR** — "click-to-open rate": of the people who opened an email, what share clicked — a more honest engagement measure than raw clicks.